Prompt Engineering Trumps Tool Selection

A 540-Test Study of AI-Generated Infrastructure Code Security

Lukas Buck
Reutlingen University - Faculty of Computer Science
Reutlingen, Baden-Württemberg, Germany
Lukas.Buck@Student.Reutlingen-University.de

Abstract

This paper presents an empirical evaluation of infrastructure-as-code generation capabilities across three AI-powered development assistants: GitHub Copilot, Cursor, and Windsurf. We conducted 540 controlled experiments using a systematic evaluation framework comprising 20 distinct infrastructure tasks, ranging from simple resource creation (achieving 99.2% ± 2.0% success) to complex multi-service architectures (35.6% ± 15.0% success). Each task was evaluated using three prompting strategies: baseline, expert-informed, and security-focused, with three runs per configuration to assess consistency.

Our methodology employs automated security analysis using tfsec and Checkov, functional validation through Terraform's native tooling, and resource completeness verification. Results demonstrate significant performance variance correlating with task complexity. Simple single-resource deployments (e.g., I2) achieved near-perfect scores across all tools (99.2% mean), while complex scenarios requiring multi-service integration (M5, I5, N5) showed substantially lower success rates (mean 42.3%). Expert prompts yielded the highest overall performance (77.9% ± 19.7%), outperforming both security-focused (73.3% ± 21.4%) and baseline prompts (66.4% ± 19.7%) across all complexity levels.

Statistical analysis reveals weak correlation between functional correctness and security scores (r=0.344-0.439), suggesting these metrics require independent validation. The coefficient of variation ranged from 0.017 for simple tasks to 0.561 for complex scenarios, indicating increasing unpredictability with task complexity. Validation success rates averaged 66.5% across all generations, with planning success at 52.9%. Notably, security-focused prompts did not consistently improve security metrics, with Copilot showing a marginal decrease (-0.6%) compared to baseline, while expert prompts improved security scores by 2.5-5.0% across tools.

Our findings indicate that AI-generated infrastructure code exhibits distinct security patterns based on task complexity and prompting strategy. The analysis identified an average of 1.2-1.6 critical and 2.8-5.2 high-severity vulnerabilities per configuration, with security scores averaging 77.6% overall but dropping below 40% for complex networking tasks. These empirical results provide a quantitative foundation for understanding the current capabilities and limitations of AI-assisted infrastructure development, particularly regarding security assurance in enterprise environments.

CCS Concepts

• **Security and privacy** → **Vulnerability management**; *Software security engineering*; • **Computing methodologies** → *Machine learning*; • **Software and its engineering** → Automatic programming; Software testing and debugging;

Keywords

AI-generated code security, Infrastructure-as-Code vulnerabilities, Prompt engineering impact, Terraform security analysis, Vulnerability detection, GitHub Copilot, Cursor, Windsurf, Comparative security evaluation, tfsec, Checkov, Cloud security automation, Empirical security assessment

I Introduction

The relentless pursuit of efficiency and scalability in modern software development has established Infrastructure-as-Code (IaC) as a foundational paradigm [4]. By leveraging machine-readable configuration files, IaC facilitates the automated provisioning and management of IT infrastructure, thereby enhancing consistency, reproducibility, and version control across diverse environments. Yet, the orchestration of complex cloud infrastructure remains a specialized and often painstaking effort, prone to manual errors, particularly in intricate, multi-cloud settings. This challenge has been significantly amplified by the recent, transformative emergence of Large Language Models (LLMs) and AI-powered coding assistants, such as the agents evaluated in this study, which have fundamentally reshaped the landscape of automated code generation in software engineering [4].

The convergence of LLMs and IaC presents an intriguing, high-leverage opportunity to mitigate the steep learning curve associated with infrastructure provisioning and empower developers to automate complex configurations with unprecedented speed. However, this revolutionary capability is accompanied by a critical and growing concern regarding the security and overall quality of the generated code [4]. These models, which are primarily trained on vast corpora of publicly available code repositories and internet-based textual data, often lack an inherent understanding of security principles, creating a significant risk of perpetuating vulnerabilities from their training data into new, production-bound codebases [4].

Empirical evidence from the scientific community starkly illustrates the magnitude of this problem. Studies have shown that a substantial fraction of LLM-generated programs exhibit security flaws; for instance, a large-scale comparison of state-of-the-art models revealed that at least 62.07% of the generated programs were vulnerable [7]. Another study, focusing on code generated by models like ChatGPT and GitHub Copilot, indicated that approximately 40% of reported cases contained insecure code [4]. These findings collectively underscore a profound challenge: while LLMs excel at generating syntactically correct and functional code, their output is not inherently secure. The limited security awareness of these models can lead to security breaches when their generated code is deployed into live production systems, positioning LLMs as a

potential new source of vulnerabilities within the software supply chain [4].

This pervasive security risk is particularly acute in the domain of IaC, where a single misconfiguration can expose an entire cloud environment. Terraform, as a leading IaC tool, is central to this discussion, as it is widely used to provision critical infrastructure components, from networking and identity management to data storage and security services. The security of Terraform code generated by AI agents is therefore not merely an academic concern, but a direct measure of the resilience of modern cloud deployments. Given the high rate of vulnerabilities in general LLM-generated code, a rigorous, systematic evaluation of AI-generated Terraform is imperative to understand the true security posture of this rapidly adopted development practice.

In this paper, we present a comprehensive security evaluation of Terraform configurations generated by three prominent AI coding agents: Copilot, Windsurf, and Cursor. Our study systematically assesses the security and functional correctness of the generated IaC across a diverse set of complex, real-world cloud provisioning tasks. The primary goal is to quantify the security risks introduced by these agents and to determine the efficacy of security-focused prompting strategies in mitigating these risks. The remainder of this paper is structured as follows. Section III details our experimental setup, evaluation metrics, and the task suite used for the assessment. Section IV presents the comparative analysis of the AI agents' performance, focusing on the trade-off between security and functionality. Finally, Section VI concludes the paper and outlines directions for future research.

II Related Work

The rapid integration of Large Language Models (LLMs) into the software development lifecycle, particularly for code generation, has catalyzed a significant body of research focused on the security implications of this paradigm shift. The literature can be broadly categorized into three areas: the general security risks of LLM-generated code, the specific challenges within Infrastructure-as-Code (IaC), and the mitigation strategies proposed by the community.

II.i Security Risks in LLM-Generated Code

A consistent finding across empirical studies is the propensity of LLMs to generate code containing security vulnerabilities [4]. This risk is attributed to the models' training on vast, publicly available code repositories that inevitably contain insecure patterns, thereby perpetuating vulnerabilities into new codebases [4]. Quantitative assessments have revealed alarming rates of insecurity; for instance, studies have found that LLM-generated programs are vulnerable in over 62% of cases [7], with specific evaluations of tools like GitHub Copilot showing vulnerable code generation in approximately 40% of instances across various vulnerability types [10]. The security quality of the output is not uniform, varying significantly depending on the programming language, and many models still fail to adopt modern security features, often relying on outdated practices, particularly in languages like C++ [2].

Compounding the issue of inherent code flaws is the human element. The introduction of AI coding assistants has been shown to foster a false sense of security among developers [1, 5]. Developers using these tools often produce code with a higher number of vulnerabilities, yet simultaneously express greater confidence in the security of their output [1]. This phenomenon highlights a critical gap in LLM capabilities, as models struggle to proactively identify and warn users about vulnerabilities in user-supplied code, with detection rates ranging from a mere 12.6% to 40% [5]. The overall picture suggests that LLMs, while powerful productivity tools, introduce a new and complex source of risk to the software supply chain [4].

II.ii Security Challenges IaC

The domain of Infrastructure-as-Code (IaC), particularly using tools like Terraform, presents a unique set of security challenges that are amplified by LLM integration. While IaC, by its nature, is intended to ensure consistent application of security policies and configurations, the scripting process does not inherently prevent misconfigurations, vulnerabilities, or privacy risks [8]. Real-world analysis of Terraform repositories demonstrates a severe prevalence of quality concerns, with over 84% of high-quality GitHub repositories containing at least one code smell that could potentially introduce security vulnerabilities [9]. Securing IaC environments, therefore, relies heavily on the practitioner's understanding and explicit adoption of security policies [8].

Recent work has begun to explore the application of LLMs to IaC, not only for generation but also for repair. Low et al. [3] proposed an LLM-based approach for Automatic Program Repair (APR) of misconfigurations in IaC. Their findings, while promising-reporting up to 84.7% reduction in misconfiguration alarms-also highlighted a significant risk: 20.4% of the suggested fixes were **hallucinated**; they passed initial misconfiguration checks but failed underlying syntax or schema validation, or did not address the root security issue [3]. This demonstrates that the application of LLMs to IaC, even for repair, requires rigorous validation.

II.iii Mitigation and Detection Strategies

The research community is actively exploring strategies to mitigate the risks associated with LLM-generated code. One key area focuses on leveraging LLMs themselves as a security tool. LLMs are emerging as transformative tools for software vulnerability detection, offering an alternative to traditional static and dynamic analysis methods [6]. They possess the capability to analyze code structure, identify complex patterns, and generate repair suggestions for vulnerability mitigation [6]. Furthermore, studies show that advanced models can be guided toward more secure code generation by providing vulnerability hints and fine-grained feedback, which helps them avoid or fix insecure patterns [10]. In the IaC domain, LLMs have been shown to significantly outperform traditional static analysis tools in detecting a broader range of code smells, especially those related to logic, which are often precursors to security vulnerabilities [9].

II.iv Contribution

While the existing literature provides a strong foundation regarding the general security challenges of LLM-generated code [2, 4, 7] and the specific misconfiguration risks in IaC [8, 9], a critical

gap remains in the **direct, comparative, and task-specific security evaluation of commercial AI agents** for generating production-ready Terraform code. Prior work often focuses on general-purpose LLMs (e.g., GPT-4) or single-agent evaluations (e.g., Copilot) on generic code tasks. Our work distinguishes itself by providing a focused, empirical evaluation of three distinct, commercially available, and domain-relevant AI agents—Copilot, Windsurf, and Cursor—on a standardized set of real-world Terraform tasks. By analyzing the security and correctness of the generated IaC, this paper offers new insights into the comparative security posture of these tools, directly informing practitioners and contributing a much-needed, task-specific benchmark to the academic community. This direct comparison of agent-generated Terraform output, coupled with a detailed analysis of the underlying security flaws, extends the work of [3] and [8] by moving beyond theoretical misconfiguration analysis to a practical, agent-centric security assessment.

III Methodology

III.i Tool Selection and Study Scope

We evaluated three commercially available AI coding assistants representing the current state of the art. GitHub Copilot operates as an IDE extension providing context-aware code suggestions. Cursor integrates large language models directly into the development environment. Windsurf represents a newer generation of AI-assisted development tools.

The evaluation focused on Terraform configurations targeting AWS infrastructure. This choice reflects Terraform's widespread adoption and AWS's dominant market position, while eliminating confounding variables from multi-platform comparison.

In order to be able to compare all tools, Claude-4-Sonnet is used as the standard model for evaluation. It is important to note that the focus is not on evaluating the model, but rather on evaluating the tools in order to identify differences and gather information about which ones are best suited to the task at hand in different areas.

III.ii Task Design and Categorization

We designed 20 infrastructure tasks spanning four domains: Networking (N), Data Storage (D), Identity and Access Management (I), and Monitoring and Security (M). Tasks cover VPCs, databases, IAM policies, security monitoring, and multi-tier architectures.

Tasks were stratified across three complexity levels: 8 easy tasks (2-4 resources), 8 medium tasks (5-10 resources with interdependencies), and 4 complex tasks (10+ resources with advanced security features).

Each task specification defined expected resource types using Terraform naming conventions (`aws_vpc`, aws-s3-bucket-encryption, etc.) and security requirements categorized as mandatory or recommended based on AWS Well-Architected Framework principles.

III.iii Prompt Engineering Strategies

We developed three prompt variants per task to investigate prompt formulation impact. The *baseline* prompt provided minimal context a single sentence describing the infrastructure objective. The *security* prompt explicitly enumerated security controls (encryption, least-privilege policies, logging, network segmentation). The *expert* prompt combined detailed specifications with best practices, including precise parameters, AWS documentation references, and resource relationships.

Each tool received identical prompts for a given task-prompt combination, enabling controlled comparison across varying instruction specificity.

III.iv Automated Evaluation Framework

For validation a Python-based evaluation pipeline for all 540 configurations (3 tools × 20 tasks × 3 prompts × 3 runs) has been implemented. The framework assessed security, functional correctness, and syntactic validity.

Security Analysis. We employed TFsec and Checkov for static security analysis. TFsec categorizes findings by severity (critical, high, medium, low). The TFsec score deducted penalties from 100: critical issues -10 points, high -5, medium -2, low -0.5. Checkov measured the percentage of passed security checks. The combined security score averaged both metrics.

Functional Correctness. We parsed Terraform configurations using regular expressions to extract declared resources. The functionality score calculated the percentage of expected resources present, providing an objective measure of task completion without subjective quality judgments.

Syntactic Validity. Each configuration underwent `terraform init`, `terraform validate`, and `terraform plan`. A binary score (0 or 100) indicated whether both validation and planning succeeded, ensuring only deployable configurations received credit.

Composite Quality Metric. The overall score combined dimensions using weighted averaging:

$$Q = 0.4 \times S_{security} + 0.4 \times S_{functionality} + 0.2 \times S_{validity} \quad (1)$$

where Q represents total quality and S denotes component scores (0–100 scale).

III.v Execution Protocol

We generated three independent runs per task-tool-prompt combination to capture variability. Tools operated in default configurations within isolated environments. We saved all configurations without manual corrections, ensuring results reflected actual tool capabilities.

III.vi Data Collection and Storage

Results were persisted to a structured CSV file with 23 fields per evaluation: task metadata, security metrics (TFsec severity counts, Checkov statistics), functionality data (expected and found resources), and validity indicators. The complete dataset comprised 540 evaluations from October 2025, with raw Terraform configurations and scan reports retained for verification.

III.vii Analysis and Visualization

Analysis employed Python scientific libraries (Pandas, NumPy, Matplotlib, Seaborn). We calculated descriptive statistics (means, standard deviations, medians, coefficient of variation) for each tool-task-prompt combination. We created diffrent visualizations including box plots, heatmaps, scatter plots, and density plots to complement quantitative findings.

III.viii Threats to Validity

Several factors limit generalizability. Tools were evaluated at a specific point in time; updates may alter performance. Task selection reflects subjective judgments about realistic scenarios. Automated functionality scoring relies on resource name matching and may miss semantic errors or incorrect configurations. Security analysis depends on TFsec and Checkov coverage, which may not encompass all concerns.

Prompt formulation introduces bias; different phrasings might yield different results. The three-run sample size may not capture rare failures. The study focused exclusively on Terraform and AWS; results may not transfer to other IaC frameworks or cloud providers. The controlled environment differs from production workflows with iterative refinement.

Additionally, the evaluation was conducted at a specific point in time (October 2025), and rapid improvements in AI models may alter these results. The evaluation framework and all data are publicly available.

IV Results

The evaluation framework processed a total of 540 individual assessments across three AI-powered coding assistants (Copilot, Cursor, and Windsurf), examining their performance on 20 distinct infrastructure-as-code tasks. Each task was executed three times with three different prompt strategies (baseline, security-focused, and expert), resulting in nine evaluations per tool per task. The comprehensive dataset encompasses security metrics from TFsec and Checkov scanners, syntactic validity measurements through Terraform validation and planning, and functional correctness scores based on expected resource presence.

Figure 1 presents the distribution of overall quality scores across all three tools. Windsurf achieved a mean score of 73.9% with a standard deviation of 20.6%, while Copilot reached 71.9% (SD: 20.3%) and Cursor attained 71.8% (SD: 21.4%). The violin plots reveal that all three tools exhibit similar distributional characteristics, with score concentrations in the upper quartile and comparable variation ranges. Windsurf demonstrates the highest median performance, though the differences between tools remain modest, with less than 2.1 percentage points separating the top performer from the third position. The Validation success rates indicate that 70.0% of Copilot's generated code passes Terraform validation, compared to 67.2% for Windsurf and 62.2% for Cursor, while planning success rates follow a similar pattern with 51.1%, 59.4%, and 48.3% respectively.

The impact of prompt engineering strategies emerges clearly in the aggregated results shown in Figure 2. Expert prompts, which include detailed architectural specifications and security requirements, consistently outperform other strategies across all tools with

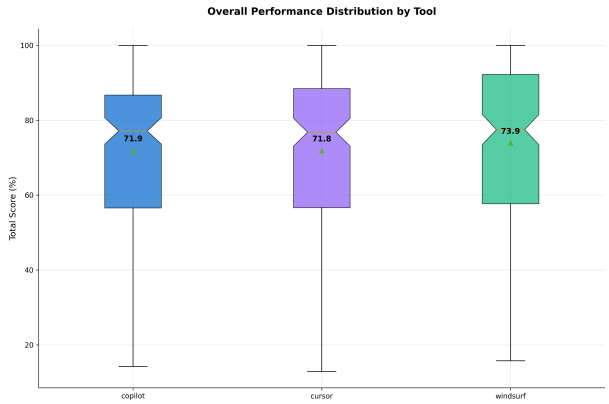

Figure 1: Overall performance distribution by tool across all 540 evaluations, showing violin plots with embedded box plots. Each distribution represents 180 evaluations combining all prompt types and task difficulties.

a mean score of 77.9% (SD: 19.7%). Security-focused prompts achieve 73.3% (SD: 21.4%), while baseline prompts, containing only minimal task descriptions, result in 66.4% (SD: 19.7%). Examining tool-specific responses to prompt strategies reveals nuanced patterns: Copilot shows the strongest improvement from baseline to expert prompts, with a 12.3 percentage point increase (from 66.6% to 78.9%), whereas Cursor exhibits an improvement of 9.6 percentage points (66.8% to 76.4%). Windsurf demonstrates a substantial 12.7 percentage point gain between baseline (65.7%) and expert prompts (78.4%), alongside a notable 11.9 percentage point improvement from baseline to security prompts (77.6%). The variability in scores, as indicated by standard deviations, remains relatively consistent across prompt types, suggesting that prompt strategy influences the central tendency of performance without fundamentally altering the consistency of tool behavior.

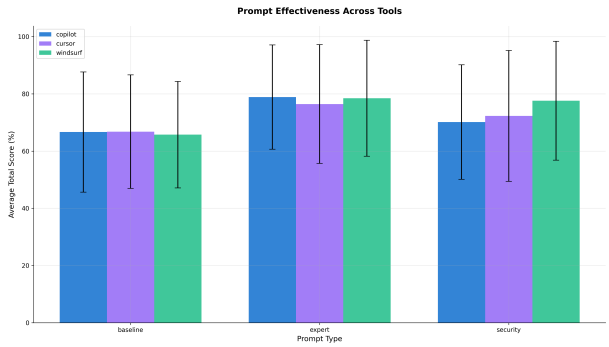

Figure 2: Mean performance scores by tool and prompt type across all task categories and difficulty levels. Error bars represent standard deviations, with n=60 evaluations per tool-prompt combination.

Figure 3 decomposes the security dimension into its constituent scanner outputs. TFsec scores, which penalize detected vulnerabilities by severity, show Cursor achieving the highest mean at 71.5%

(SD: 34.2%), followed by Windsurf at 69.6% (SD: 35.6%) and Copilot at 68.1% (SD: 36.3%). Checkov results, representing the percentage of passed compliance checks, demonstrate more consistent performance across tools, with Windsurf leading at 85.8% (SD: 15.0%), Cursor at 85.6% (SD: 17.1%), and Copilot at 84.8% (SD: 16.8%). The combined security scores, calculated as the arithmetic mean of TFsec and Checkov metrics, place Cursor marginally ahead at 78.6%, with Windsurf at 77.7% and Copilot at 76.5%. Vulnerability distribution patterns reveal that critical severity issues average 1.6 per evaluation for Copilot, 1.4 for Cursor, and 1.2 for Windsurf, while high-severity findings show more pronounced differences with 5.2, 2.8, and 3.1 occurrences respectively. The substantial standard deviations in TFsec scores indicate high variability in security vulnerability detection across different tasks and configurations.

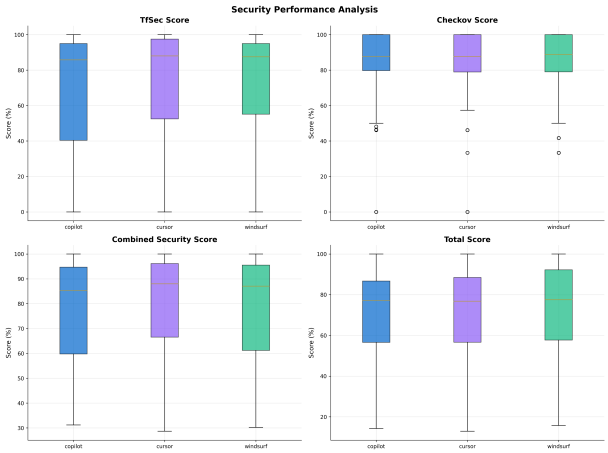

Figure 3: Detailed security performance metrics showing TFsec scores, Checkov scores, combined security scores, and average vulnerability counts by severity level for each tool.

The task complexity heatmap in Figure 4 illustrates performance variations across the 20 individual tasks, organized by category and difficulty level. Task I2 (IAM Password Policy) emerges as the most successfully addressed challenge, with a mean score of 99.2% across all tools and prompt types, followed by I1 (IAM Role for EC2) at 94.3%. Conversely, task M5 (Security Operations Center Automation) proves most challenging, yielding a mean score of only 35.6%, with I5 (Identity Federation with SAML) at 43.4% and N5 (Multi-Tier Architecture) at 47.9% representing the next most difficult tasks. Category-level analysis reveals that Identity and Access Management tasks achieve the highest average performance at 81.3%, while Networking and Infrastructure tasks present the greatest challenges at 63.7%. Data Storage and Monitoring tasks fall in intermediate positions with 74.5% and 70.7% respectively. The heatmap demonstrates relatively consistent tool behavior within easy-level tasks, where performance differences remain within 5-10 percentage points, while complex-level tasks exhibit greater variance in tool capabilities, with differences occasionally exceeding 15 percentage points for individual tasks.

Consistency analysis, presented in Figure 5, employs the coefficient of variation to assess performance stability across the three

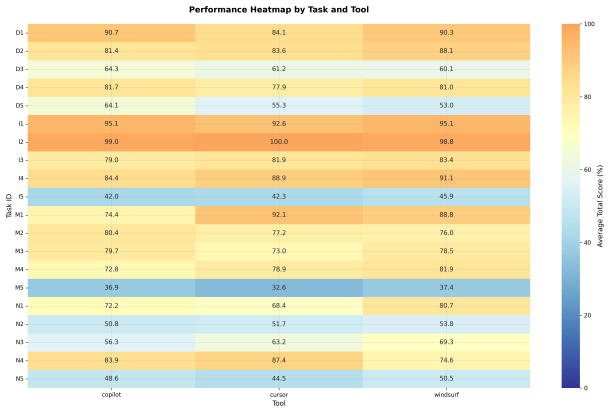

Figure 4: Performance heatmap showing mean scores for each tool-task combination. Tasks are grouped by category (D: Data, I: IAM, M: Monitoring, N: Network) and arranged by difficulty level, with color intensity representing performance from 30% (red) to 100% (green).

independent runs for each tool-task-prompt combination. Windsurf exhibits the lowest average coefficient of variation at 0.158 (SD: 0.084), indicating the most consistent behavior, while Copilot (CV: 0.176, SD: 0.095) and Cursor (CV: 0.178, SD: 0.122) demonstrate slightly higher variability. Task-level analysis identifies I2 as the most consistently executed task with a coefficient of variation of 0.017, reflecting near-identical performance across runs, whereas M5 shows the highest variability at 0.433, indicating substantial performance fluctuations. The distribution of coefficients across tasks reveals that simple IAM-related tasks (I1, I2, I3) consistently exhibit low variability (CV < 0.09), while complex monitoring and networking tasks (M5, N5, N2) demonstrate high variability (CV > 0.25). Tool-specific patterns show that Windsurf maintains superior consistency across difficult tasks, particularly evident in tasks D3, I3, and M5, where its coefficient of variation remains substantially below that of Copilot and Cursor.

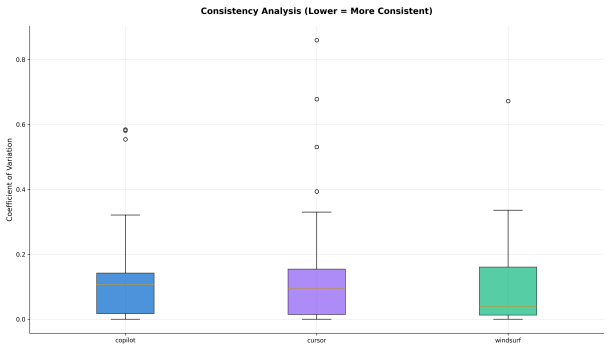

Figure 5: Consistency analysis showing coefficient of variation for each tool across all tasks. Lower values indicate more consistent performance across the three independent runs. Box plots show distribution with median, quartiles, and outliers.

The relationship between security and functionality scores appears in Figure 6, which examines potential trade-offs between these two quality dimensions. Correlation coefficients indicate positive relationships for all tools, with Cursor showing the strongest correlation at 0.439, followed by Windsurf at 0.372 and Copilot at 0.344. Quadrant analysis, dividing evaluations at median thresholds, reveals that Windsurf achieves the highest proportion of cases with both high security and high functionality at 38.3%, compared to 36.1% for both Copilot and Cursor. The proportion of evaluations demonstrating high security with low functionality ranges from 20.6% for Copilot and Windsurf to 23.9% for Cursor. Cases exhibiting low security but high functionality occur less frequently, representing 20.0%, 17.8%, and 15.0% for Copilot, Cursor, and Windsurf respectively. The scatter plots demonstrate substantial variance in both dimensions, with security scores spanning from approximately 20% to 100% and functionality scores showing similar ranges, though the concentration of data points in the upper-right quadrant indicates that strong performance in one dimension often coincides with strong performance in the other.

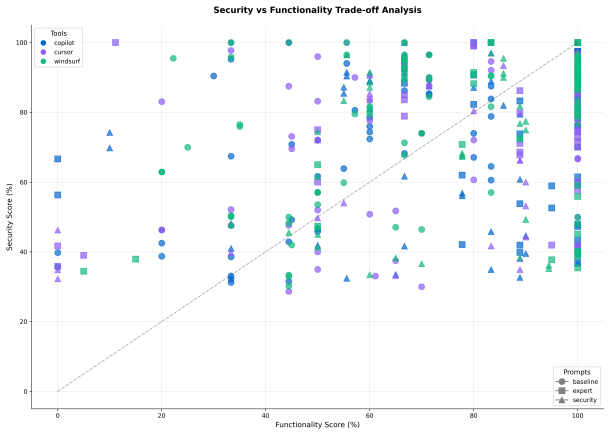

Figure 6: Scatter plots showing the relationship between security and functionality scores for each tool. Each point represents one evaluation, with quadrant divisions at median values. Correlation coefficients indicate the strength of the relationship.

The impact of security-focused prompts on actual security outcomes emerges in Figure 8. Baseline prompts result in mean security scores of 75.8% for Copilot, 76.1% for Cursor, and 74.7% for Windsurf. Security-specific prompts produce divergent effects across tools: Copilot exhibits a slight decrease to 75.2% (a negative impact of 0.6 percentage points), while Cursor improves to 78.4% (+2.3 percentage points) and Windsurf advances to 79.1% (+4.4 percentage points). Expert prompts, which incorporate security requirements within comprehensive architectural specifications, achieve security scores of 78.3%, 81.1%, and 79.3% for Copilot, Cursor, and Windsurf respectively, representing improvements of 2.5, 5.0, and 4.5 percentage points over baseline. Functionality scores show more consistent patterns, with security prompts improving functionality by 18.5, 13.1, and 21.9 percentage points across the three tools compared to baseline approaches. Expert prompts further enhance

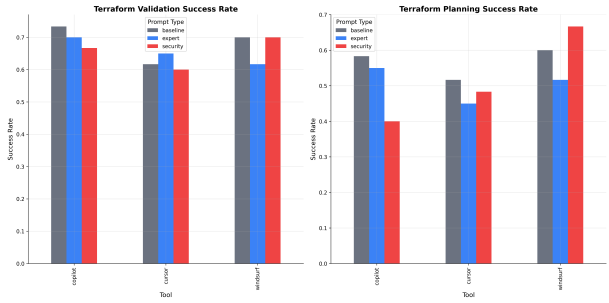

Figure 7: Validation and planning success rates broken down by tool and prompt type. The gap between validation and planning success indicates the proportion of syntactically valid code that fails logical consistency checks.

functionality, though the magnitude of improvement varies by tool, suggesting that comprehensive prompts benefit both security and functional dimensions simultaneously.

Figure 8: Comparison of security and functionality scores across prompt types, with baseline performance as reference point. Bars show absolute scores, with annotations indicating percentage point differences from baseline.

Task difficulty ranking in Figure 9 organizes all 20 tasks by mean performance across tools and prompts. The distribution reveals a clear stratification, with easy-level tasks predominantly occupying the top positions, medium-level tasks distributed throughout the middle range, and complex-level tasks consistently appearing at the bottom. The performance gap between the highest-scoring task (I2 at 99.2%) and the lowest-scoring task (M5 at 35.6%) spans 63.6 percentage points, illustrating substantial variation in task difficulty.

Standard deviations for each task indicate that complex tasks not only achieve lower mean scores but also exhibit higher variability (M5 SD: 15.0%, I5 SD: 10.0%, N5 SD: 13.9%), while top-performing easy tasks demonstrate both high scores and low variability (I2 SD: 2.0%, I1 SD: 7.7%, D1 SD: 8.0%). The divergence between security and functionality scores becomes particularly pronounced in complex tasks: N5 shows a functionality score of 73.0% against a security score of only 37.4%, suggesting that tools successfully generate required resources but struggle to implement proper security controls for complex network architectures. Conversely, task

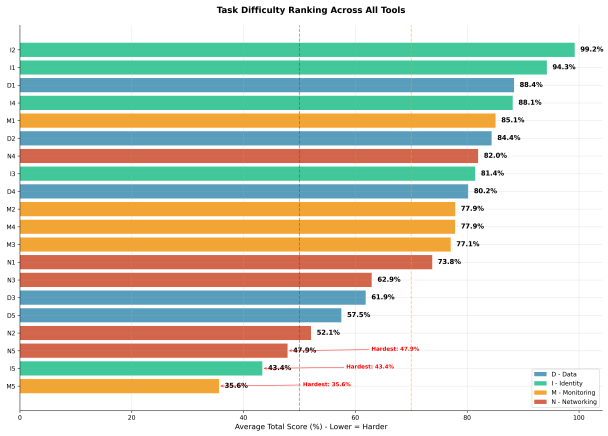

Figure 9: Complete task ranking from most challenging to least challenging, based on mean performance across all tools and prompts. Each bar shows the combined score with annotations for security and functionality components. Error bars represent standard deviations across 27 evaluations per task (3 tools × 3 prompts × 3 runs).

M3 demonstrates high security (90.8%) with moderate functionality (74.1%), indicating that security implementations for AWS Config compliance prove more achievable than generating all expected monitoring resources.

Statistical significance testing through one-way ANOVA on overall quality scores yields an F-statistic of 1.847 with a p-value of 0.159, indicating that the observed differences in mean performance between the three tools do not reach statistical significance at the $\alpha = 0.05$ level. Pairwise t-tests confirm this finding, with Windsurf versus Copilot yielding $t = 1.348$ ($p = 0.178$), Windsurf versus Cursor producing $t = 1.535$ ($p = 0.125$), and Copilot versus Cursor resulting in $t = 0.069$ ($p = 0.945$). Cohen's d effect sizes remain small for all comparisons, ranging from 0.006 to 0.099, indicating negligible practical significance despite the observed differences in mean scores. Prompt type comparisons within each tool demonstrate statistically significant effects: for Copilot, expert prompts significantly outperform baseline prompts ($t = 3.892$, $p < 0.001$, $d = 0.633$), and security prompts show improvement over baseline ($t = 1.151$, $p = 0.252$, $d = 0.187$, not significant). Similar patterns emerge for Cursor and Windsurf, with expert versus baseline comparisons yielding p-values below 0.01 for both tools, confirming that prompt engineering substantially influences performance outcomes across all examined AI assistants.

V Discussion

Several patterns emerge from the empirical findings in Section IV that warrant closer examination, particularly regarding tool capability convergence, asymmetric impacts of prompt engineering strategies, and relationships between task complexity and security outcomes. Statistical analysis reveals no significant differences in overall performance among the three AI-powered coding assistants ($F = 1.847$, $p = 0.159$), yet examining the distribution of strengths across security, functionality, and consistency dimensions uncovers more nuanced variations in underlying capabilities.

A mean performance spread of merely 2.1 percentage points separates Windsurf (73.9%), Copilot (71.9%), and Cursor (71.8%). This narrow margin suggests field maturation, where competing tools converge toward similar baseline competencies through comparable training methodologies and model architectures. Such convergence, however, does not necessarily imply functional equivalence across all operational dimensions. Windsurf's validation-to-planning conversion rate of 88.4% substantially exceeds both Copilot (72.9%) and Cursor (77.7%), pointing toward potential differences in how these systems represent and reason about infrastructure dependencies and resource relationships. These distinctions surface through multi-stage evaluation rather than aggregate scoring alone.

Prompt engineering emerges as the most substantial source of performance variation in the dataset. Expert prompts yield improvements of 9.6 to 12.7 percentage points over baseline approaches across all tools. Notably, these gains exceed the performance differences between tools by factors of approximately five to six, raising questions about the relative importance of prompt optimization versus tool selection in practical applications.Multiple pathways appear to underlie these improvements. Explicit security requirements directly influence the inclusion of protective measures such as encryption configurations and access restrictions, while comprehensive architectural specifications reduce ambiguity and constrain the solution space toward implementations that align with established infrastructure patterns.

Security-focused prompts, however, demonstrate divergent effects across tools. Copilot exhibits a slight negative impact on security scores (-0.6 percentage points) while Cursor (+2.3 percentage points) and particularly Windsurf (+4.4 percentage points) show substantial improvements. Such divergence points toward fundamental differences in how these systems process and prioritize security-related instructions, potentially reflecting variations in training data composition, reward modeling approaches, or instruction-following architectures. Expert prompts, which integrate security requirements within broader architectural context, consistently outperform isolated security prompts across all tools, suggesting that security considerations may benefit from embedding within holistic specifications rather than presentation as standalone constraints.

Systematic patterns in task difficulty expose potential limitations in current AI-assisted code generation capabilities. A 63.6 percentage point performance spread separates the most successful task (I2: IAM Password Policy at 99.2%) from the most challenging (M5: Security Operations Center Automation at 35.6%), indicating that task complexity may serve as a stronger predictor of success than either tool selection or prompt strategy.

Category-level analysis reveals substantial performance differences, with Identity and Access Management tasks achieving 81.3% compared to 63.7% for Networking and Infrastructure tasks. This disparity might reflect both the relative simplicity of IAM resource declarations in Terraform syntax and the abundance of similar examples in training corpora, given the ubiquity of IAM configurations in public infrastructure-as-code repositories.

Complex networking tasks present combinatorial challenges in correctly specifying interdependent resources such as VPCs, subnets, route tables, and security groups, where logical inconsistencies in any single relationship can cascade into validation or planning failures. Task M5 exemplifies these orchestration challenges. Combining CloudWatch Events, Lambda functions, SNS topics, and IAM policies in an integrated monitoring pipeline requires not only syntactically correct individual components but also appropriate connections and permissions across the entire system.

Positive correlations between security and functionality scores (ranging from 0.344 to 0.439) present an interesting counterpoint to common assumptions about security-functionality trade-offs. Rather than the anticipated inverse relationship where security hardening compromises functionality or functional requirements necessitate security compromises, the data reveal that successful implementations tend to excel in both dimensions simultaneously while poor implementations struggle across all quality aspects.

One interpretation of this pattern positions the AI assistant's fundamental comprehension of task requirements as a shared determinant of both security and functionality. When tools successfully interpret the problem space, they appear to generate configurations that naturally incorporate both required resources and appropriate security controls. Conversely, when task understanding fails, the resulting code exhibits deficiencies across multiple quality dimensions. Quadrant analysis shows that 36-38% of evaluations achieve both high security and high functionality, while only 15-20% demonstrate high functionality with low security, further supporting the interpretation that these dimensions may reflect shared underlying competence rather than adversarial objectives. Output stability patterns reveal task complexity as a primary factor influencing variation across independent runs. Simple IAM tasks (I1, I2, I3) demonstrate remarkably low coefficients of variation ($CV < 0.09$), with tools producing nearly identical outputs when presented with the same prompts multiple times. Complex tasks (M5, N5, N2), conversely, exhibit substantial variability ($CV > 0.25$), suggesting these challenges may reside near the boundaries of current capabilities where stochastic elements in the generation process produce divergent solution paths.

Windsurf exhibits lower variability (CV: 0.158) compared to Copilot (0.176) and Cursor (0.178), particularly on difficult tasks, potentially indicating more constrained generation strategies or stronger regularization mechanisms. High consistency on routine tasks could enable reliable automation of standard infrastructure patterns, while elevated variability on complex tasks raises questions about the reliability of these tools for novel or intricate infrastructure requirements without careful review and validation procedures.

VI Conclusion

This empirical evaluation of three AI-powered coding assistants across 540 infrastructure-as-code generation tasks reveals a landscape characterized by converging capabilities, substantial prompt sensitivity, and task-specific performance variations. Statistical analysis indicates no significant overall performance differences between Windsurf, Copilot, and Cursor ($F = 1.847$, $p = 0.159$), with mean scores separated by merely 2.1 percentage points. Yet beneath this apparent parity lie meaningful distinctions in validation-to-planning conversion rates, security posture generation, and output consistency across different task complexities.

Prompt engineering emerges as the dominant factor influencing generation quality, exceeding tool selection effects by factors of five to six. Expert prompts incorporating architectural context and security requirements yield improvements of 9.6 to 12.7 percentage points over baseline approaches, suggesting that how practitioners communicate requirements to these systems may matter more than which system they choose. Security-focused prompts demonstrate divergent effects across tools, with Windsurf showing the strongest positive response while Copilot exhibits slight negative impacts, indicating fundamental differences in how these systems process and prioritize security-related instructions.

Task complexity proves a stronger predictor of success than either tool selection or prompt strategy. A 63.6 percentage point performance spread separates the most successful task from the most challenging, with Identity and Access Management configurations achieving substantially higher success rates than complex networking and monitoring scenarios. This pattern reflects both the relative simplicity of certain infrastructure patterns and their prevalence in training data, while exposing current limitations in multi-resource orchestration where logical dependencies between components must be correctly established.

The positive correlation between security and functionality scores challenges conventional assumptions about inherent trade-offs between these dimensions. Successful implementations tend to excel in both aspects simultaneously, suggesting that a shared underlying competence in task comprehension drives quality across multiple dimensions rather than requiring explicit balancing of competing objectives.

Several observations bear relevance for practitioners integrating AI-assisted infrastructure-as-code generation into development workflows. The substantial impact of prompt engineering suggests that organizations may benefit from investing in structured prompt templates and guidelines as much as, if not more than, focusing primarily on vendor evaluation. Integrating security requirements within comprehensive architectural specifications consistently outperforms presenting them as standalone constraints, indicating that context-rich prompts enable more effective reasoning about appropriate implementations.

Specific task categories where all examined tools demonstrate elevated failure rates, particularly complex networking and monitoring scenarios achieving only 63.7% and 70.7% success rates respectively, warrant enhanced validation procedures and potentially human oversight rather than automated acceptance of generated configurations. The gap between syntactic validation success and planning success, where 11.6% to 27.1% of syntactically valid code fails logical consistency checks, highlights that validation alone provides insufficient quality assurance. Incorporating full Terraform planning stages into quality gates addresses this limitation despite additional computational overhead.

The observed positive correlation between security and functionality raises questions about the necessity of maintaining separate evaluation tracks for these dimensions in code review processes. Identifying weak implementations through integrated quality metrics may prove more efficient than parallel security and functionality review workflows, given that high-performing configurations tend to excel in both dimensions while poor implementations struggle across all quality aspects.

Methodological limitations circumscribe the scope of these findings. The evaluation examines 20 tasks across four infrastructure categories using AWS-specific Terraform configurations, necessarily capturing only a subset of production scenarios and preventing extrapolation to multi-cloud strategies or alternative infrastructure-as-code tools. Static security analysis through TFsec and Checkov captures configuration-level vulnerabilities but cannot assess runtime security characteristics or defense-in-depth properties that emerge in deployed systems. Functionality assessment based on expected resource presence, while objectively quantifiable, does not evaluate whether generated configurations achieve intended operational outcomes when actually deployed.

The convergence of these three major AI coding assistants toward similar baseline performance levels, combined with their shared sensitivity to prompt engineering and task complexity, suggests that the field has reached a degree of maturation while simultaneously revealing persistent challenges in complex multi-resource orchestration scenarios. As these tools continue evolving, understanding the conditions under which they excel or struggle, the mechanisms through which prompt strategies influence output quality, and the potential for more sophisticated validation and review workflows will prove essential for their effective integration into infrastructure development practices.

Acknowledgments

This independent research was conducted for academic purposes. All evaluations represent the author's findings as of October 2025. The author used Windsurf Cascade, Cursor and GitHub Copilot via VisualSudioCode for the IaC evaluation and DeepL for language assistance.

Data and Code Availability

Repository: https://github.com/L4XB/ai-iac-security-study-preview

References

[1] Obed Asare, Yicheng Li, Ling Zhang, Xiaokuan Zhang, and Ziyu Zhang. 2024. A User-centered Security Evaluation of Copilot. (2024), 320–331. https://doi.org/10.1145/3597503.3639154

[2] Mohammed F. Kharma, Soohyeon Choi, Mohammad Alkhanafseh, and David Mohaisen. 2025. Security and Quality in LLM-Generated Code: A Multi-Language, Multi-Model Analysis. *arXiv preprint arXiv:2502.01853* (2025).

[3] En Low, Carmen Cheh, and Binbin Chen. 2024. Repairing Infrastructure-as-Code using Large Language Models. In *2024 IEEE Secure Development Conference (SecDev)*. to appear. https://doi.org/10.1109/SecDev61168.2024.10734039

[4] Ahmad Mohsin, Helge Janicke, Adrian Wood, Iqbal H. Sarker, Leandros Maglaras, and Naeem Janjua. 2024. Can We Trust Large Language Models Generated Code? A Framework for In-Context Learning, Security Patterns, and Code Evaluations Across Diverse LLMs. *arXiv preprint arXiv:2406.12513* (2024). https://arxiv.org/abs/2406.12513

[5] Amirali Sajadi, Binh Le, Anh Nguyen, Kostadin Damevski, and Preetha Chatterjee. 2025. Do LLMs consider security? an empirical study on responses to programming questions. *Empirical Software Engineering* 30 (2025), 101. Issue 1. https://doi.org/10.1007/s10664-025-10658-6

[6] Ze Sheng, Zhicheng Chen, Shuning Gu, Heqing Huang, Guofei Gu, and Jeff Huang. 2025. LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights. *Comput. Surveys* (2025). https://doi.org/10.1145/3769082

[7] N. Tihanyi, M. Bisztray, L. Varga, and D. Varró. 2025. How secure is AI-generated code: A large-scale comparison of large language models. *Software Quality Journal* 33 (2025), 301–329. Issue 2. https://doi.org/10.1007/s10664-024-10590-1

[8] Alexandre Verdet, Mohammad Hamdaqa, Leuson Da Silva, and Foutse Khomh. 2025. Assessing the adoption of security policies by developers in terraform across different cloud providers. *Empirical Software Engineering* 30 (2025). Issue 74. https://doi.org/10.1007/s10664-024-10610-0

[9] Quoc-Huy Vo, Ha Dao, and Kensuke Fukuda. 2025. Harnessing the Power of LLMs for Code Smell Detection in Terraform Infrastructure as Code. In *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*. to appear. https://doi.org/10.1109/COMPSAC.2025.11126823

[10] Hao Yan, Swapneel Suhas Vaidya, Xiaokuan Zhang, and Ziyu Yao. 2025. Guiding AI to Fix Its Own Flaws: An Empirical Study on LLM-Driven Secure Code Generation. *arXiv preprint arXiv:2506.23034* (2025). https://arxiv.org/abs/2506.23034